HDOC Day-62

Merge Two Sorted Arrays Using Pointers

1. Problem Understanding

Write a C program that reads the sizes and elements of two sorted variable-length integer arrays, arr1[] and arr2[], in ascending order. Pass both arrays to a function named merge_sorted_arrays() using their addresses. Use pointers to pass the arrays between the calling function and merge_sorted_arrays().

- Merge arr1[] and arr2[] into a third array, merged[].
- Maintain ascending order in the merged array.
- Preserve duplicate elements.
- Display the merged elements separated by spaces.
- Use the two-pointer technique to merge the arrays efficiently.

2. Standard Test Cases

Test Case	Size 1	Array 1 Input	Size 2	Array 2 Input	Expected Output
1	4	1 3 5 7	4	2 4 6 8	1 2 3 4 5 6 7 8
2	4	1 2 2 4	3	2 3 5	1 2 2 2 3 4 5
3	3	1 4 7	5	2 3 5 6 8	1 2 3 4 5 6 7 8
4	3	1 1 3	4	1 2 2 5	1 1 1 2 2 3 5
5	2	2 6	3	1 3 7	1 2 3 6 7

3. Algorithm

1. Read the size n1 and elements of the first sorted array arr1[].
2. Read the size n2 and elements of the second sorted array arr2[].
3. Create a third array merged[] of size n1 + n2.
4. Pass arr1, arr2 and merged to merge_sorted_arrays() using their addresses.
5. Initialize three indices i = 0, j = 0 and k = 0.
6. Compare arr1[i] and arr2[j]. Store the smaller element in merged[k] and move the corresponding pointer forward.
7. Continue until one of the two input arrays is completely processed.
8. Copy all remaining elements of the unfinished array into merged[].
9. Display all elements of merged[] separated by spaces.

4. C Program

```
#include <stdio.h>
```

```
void merge_sorted_arrays(int *arr1, int n1, int *arr2, int n2, int *merged)
{
  int i = 0, j = 0, k = 0;

  while (i < n1 && j < n2)
  {
    if (*(arr1 + i) <= *(arr2 + j))
    {
      *(merged + k) = *(arr1 + i);
      i++;
    }
    else
    {
      *(merged + k) = *(arr2 + j);
      j++;
    }
    k++;
  }

  while (i < n1)
  {
    *(merged + k) = *(arr1 + i);
    i++;
    k++;
  }

  while (j < n2)
  {
    *(merged + k) = *(arr2 + j);
    j++;
    k++;
  }
}

int main()
{
  int n1, n2, i;

  printf("Enter size of first array: ");
  scanf("%d", &n1);
  int arr1[n1];

  printf("Enter elements of first sorted array: ");
  for (i = 0; i < n1; i++)
    scanf("%d", &arr1[i]);

  printf("Enter size of second array: ");
  scanf("%d", &n2);
  int arr2[n2];
  int merged[n1 + n2];
```

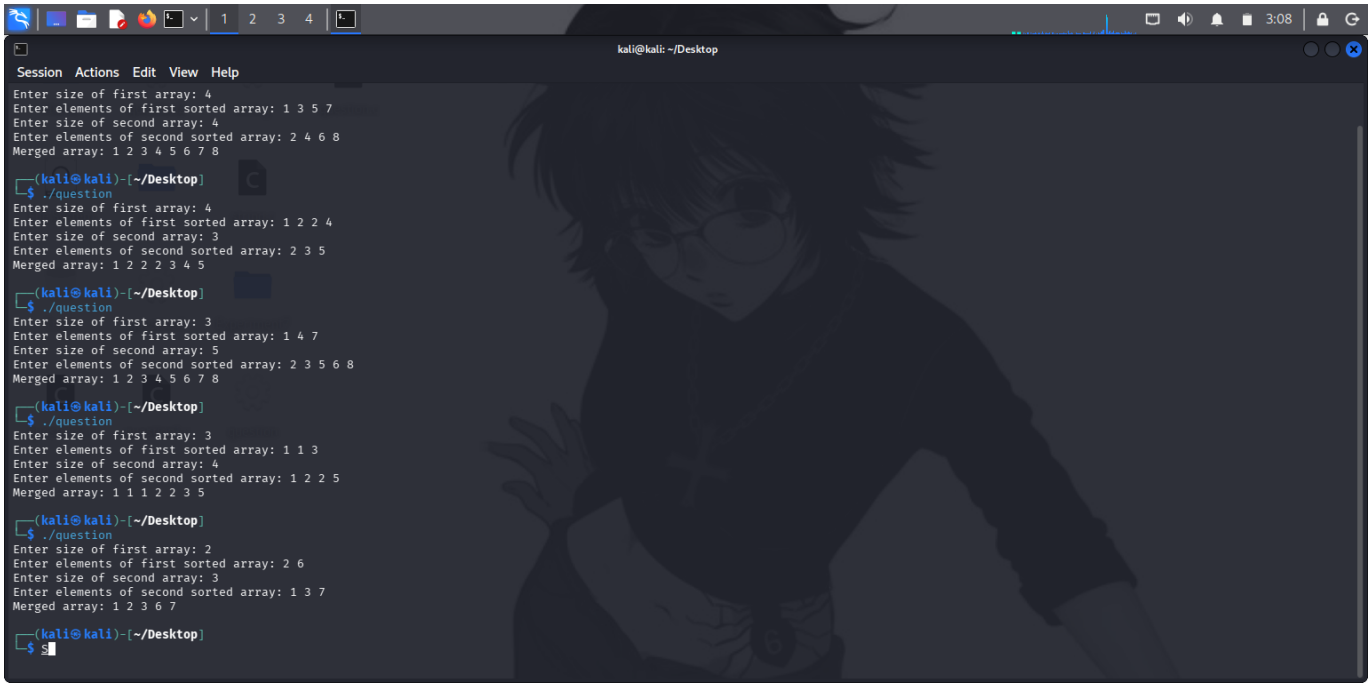
```
    printf("Enter elements of second sorted array: ");
    for (i = 0; i < n2; i++)
        scanf("%d", &arr2[i]);

    merge_sorted_arrays(arr1, n1, arr2, n2, merged);

    printf("Merged array: ");
    for (i = 0; i < n1 + n2; i++)
        printf("%d ", merged[i]);

    return 0;
}
```

5. Compilation / Execution Evidence



6. Verification

Test Case	Size 1	Array 1	Size 2	Array 2	Expected Output	Actual Output	Result
1	4	1 3 5 7	4	2 4 6 8	1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8	Pass
2	4	1 2 2 4	3	2 3 5	1 2 2 2 3 4 5	1 2 2 2 3 4 5	Pass
3	3	1 4 7	5	2 3 5 6 8	1 2 3 4 5 6 7 8	1 2 3 4 5 6 7 8	Pass
4	3	1 1 3	4	1 2 2 5	1 1 1 2 2 3 5	1 1 1 2 2 3 5	Pass
5	2	2 6	3	1 3 7	1 2 3 6 7	1 2 3 6 7	Pass